

NASGuard: A Novel Accelerator Architecture for Robust Neural Architecture Search (NAS) Networks

Xingbin Wang^{*†}, Boyan Zhao^{*}, Rui Hou^{*‡}, Amro Awad[§], Zhihong Tian[¶], Dan Meng^{*}

^{*}State Key Laboratory of Information Security, Institute of Information Engineering, CAS, Beijing, China

[†]School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China

[§]NC State University, Raleigh, USA

[¶]Guangzhou University, Guangzhou, China

{wangxingbin, zhaoboyan, hourui, mengdan}@iie.ac.cn, ajawad@ncsu.edu, tianzhihong@gzhu.edu.cn

Abstract—Due to the wide deployment of deep learning applications in safety-critical systems, robust and secure execution of deep learning workloads is imperative. Adversarial examples, where the inputs are carefully designed to mislead the machine learning model is among the most challenging attacks to detect and defeat. The most dominant approach for defending against adversarial examples as a key metric of the neural network's robustness. While NAS has been proven effective in improving the robustness (and accuracy in general), the NAS-generated network models run noticeably slower on typical DNN accelerators than the hand-crafted networks, mainly because DNN accelerators are not optimized for robust NAS-generated models. In particular, the inherent multi-branch nature of NAS-generated networks causes unacceptable performance and energy overheads.

To bridge the gap between the robustness and performance efficiency of deep learning applications, we need to rethink the design of AI accelerators to enable efficient execution of robust (auto-generated) neural networks. In this paper, we propose a novel hardware architecture, NASGuard, which enables efficient inference of robust NAS networks. NASGuard leverages a heuristic multi-branch mapping model to improve the efficiency of the underlying computing resources. Moreover, NASGuard addresses the load imbalance problem between the computation and memory-access tasks from multi-branch parallel computing. Finally, we propose a topology-aware performance prediction model for data prefetching, to fully exploit the temporal and spatial localities of robust NAS-generated architectures. We have implemented NASGuard with Verilog RTL. The evaluation results show that NASGuard achieves an average speedup of $1.74\times$ over the baseline DNN accelerator.

Index Terms—Robust NAS network, adversarial example, DNN accelerator

I. INTRODUCTION

With the widespread deployment of deep learning applications in safety-critical systems, attacks that could potentially deter safe adoption of deep learning applications have been studied extensively [1]–[10]. Adversarial examples that are carefully crafted to misguide highly-accurate deep learning models are considered among the most challenging ones. Various systematic defense techniques have been proposed to generate neural network architectures with robustness as the

primary optimization function to defend against adversarial example attacks [11]–[17]. Such systematic approaches are commonly known as *Neural Architecture Search (NAS)* [18]. To generate robust models with NAS, network architectures and parameters are exhaustively searched for improved robustness of machine learning models. Table I summarizes the recent approaches for robust NAS networks.

TABLE I: Typical robust NAS networks

Model	Methods against adversarial attacks	Source
RobNet [11]	FSGM [19], PGD100 [20], DeepFool [21]	CVPR'2020
RASNet [12]	ILLC [5], One-pixel [22], ATNs [23]	GECCO'2020
S-DARTS [24]	FGSM [19], PGD [20], SDARTS [24]	ICML'2020
ABanditNAS [25]	MI-FGSM [19], PGD-40 [20], BIM [5]	ECCV'2020
RAS [26]	FGSM [19], PGD [20], One-pixel [22]	AISeC'2020
DNHA [27]	R-FGSM [19], PGD [20], StepLL [5]	arXiv'2020
RBF-CNN [28]	MI-FGSM [19], C&W [29], EAD [30]	arXiv'2020
RACL [31]	FGSM [19], MIM [32], PGD-100 [20]	arXiv'2020

In contrast with traditional deep neural networks (DNNs) [33]–[36], robust NAS networks exhibit significantly different network topologies and dataflows. First, NAS primarily searches for *computational cells*, each of which contains multiple branches, and uses them as the basic building blocks. Second, these network branches contain complex and highly concurrent dataflows, including inter-branch communication and backward dataflows. Finally, these networks usually use many intra/inter-cell shortcut and dense connections to realize cross-layer data communication.

Such significant differences in network structure pose challenges to existing DNN accelerator designs. Current DNN accelerators are designed for common DNN networks [37]–[40] rather than robust NAS networks, resulting in inefficient inference of such new networks. The inefficiencies mainly stem from the following aspects. First, typical industrial DNN accelerators, such as TPU [37] and NVDLA [41], execute each branch serially for the robust NAS networks with cell-based multi-branch fabrics, and have failed to support efficient parallel execution. Second, the characteristics of single-branch network architectures (e.g., Depth-wise convolution (DWconv) and nonlinear computation) prevent them from fully utilizing on-chip computing resources and memory bandwidth. Finally, most DNN accelerator architectures from academia, such as

[†]Corresponding author: Rui Hou (E-mail: hourui@iie.ac.cn)

Diannao [38] and Eyeriss [39], also adopt the design of serial execution of branches, and cannot fully exploit the data locality across multiple branches or fully reuse internal data during inference.

Our work lies at the intersection of AI algorithms, security, and DNN acceleration. The main goal is to design a new accelerator for robust NAS networks that can fully utilize the accelerator’s computation resources and improve data locality. To the best of our knowledge, our paper is the first to propose a designated accelerator architecture for robust NAS networks. Our architecture, *NASGuard*, can fully exploit the fine-grained parallelism of multiple branches and dataflows at the hardware level, while maintaining all data communication and synchronization requirements, and thus enable efficient execution of the networks. Through our deep analyses of robust NAS networks, we observe that they have common network and computing characteristics. We accordingly propose a robust NAS network inference accelerator based on a many-core architecture to support parallel computation of multiple branches within each cell.

The main contributions of this paper are as follows.

- **Propose an accelerator architecture for robust NAS networks.** Our paper is the first to discuss and analyze the limitations of current DNN accelerators when used for robust NAS networks, a commonly used mechanism for defending adversarial example attacks. Guided by the analysis, we design a new accelerator architecture that can efficiently support the inference of robust NAS networks.
- **Design a multi-branch mapping model for proper allocation of computation and storage resources.** Multi-branch scheduler uses a hardware mapping table generated by a heuristic multi-branch mapping model to allocate multi-branch computation resources on *NASGuard*. Doing so can alleviate the data traffic congestion, shorten the communication distances, and consequently improve data reuse and reduce data copy operations.
- **Employ a multi-granularity dynamic scheduling mechanism to alleviate load imbalance.** A two-level scheduling mechanism with different granularities is proposed. It can achieve computational load balance across branches and avoid performance bottleneck in network communication.
- **Implement hardware prefetch operations to fully exploit the temporal and spatial localities of robust NAS networks.** A branch monitor is proposed to use the prediction information generated by the topology-aware prediction model to prefetch the weights of the next branch (forward or backward), as well as the activation values of previous layers. It can significantly improve the performance of inference.

To evaluate *NASGuard* architecture, we have used a number of representative robust NAS network workloads. The experimental results show that *NASGuard* successfully utilizes its processing element (PE) groups and memory bandwidth, while reducing the requirement on the prefetch buffer capacity. The

improved resource utilization enables *NASGuard* to achieve $1.74\times$ speedup on average over the baseline architecture while exhibiting greater scalability.

The rest of the paper is organized as follows. In Section II, we summarize common features of robust NAS networks and weaknesses of current DNN accelerators. Section III and Section IV present the overview of *NASGuard* and its compiler, respectively. Section V and Section VI depict the multi-branch mapping model and the multi-granularity scheduling mechanism, respectively. Section VII introduces the performance prediction model and prefetching mechanism. Section VIII evaluates the performance and energy consumption of *NASGuard*. Section IX is the discussion, and the related works are introduced in Section X. Section XI concludes this paper.

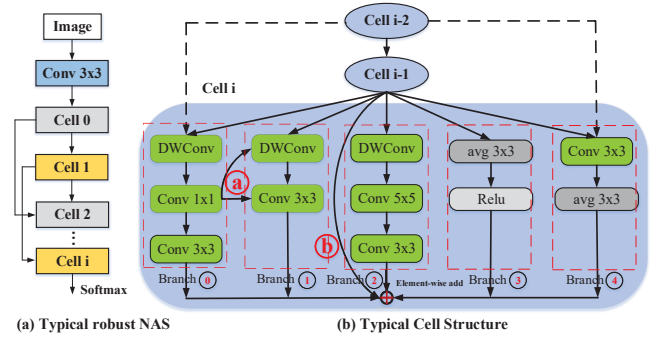


Fig. 1: Typical structure of a robust NAS network and Cell. Cell in (b) contains five branches. Branch ① is composed of one depth-wise convolution layer and two convolution layers: ④ represents the example of a branch jump of multiple dataflows; ⑤ shows an example of cross-layer data communication.

II. BACKGROUND AND MOTIVATION

A. Common Features of Robust NAS Networks

The search of robust NAS networks is usually built upon a hierarchy of cells, as shown in Figure 1(a). Each cell contains multiple branches, and operations of each individual branch (convolution (Conv2d), depth-wise convolution, ReLU, pooling, etc) are different. There are also possible interweaving dataflows among different branches. The quantitative analyses of typical robust NAS networks in Table I shows the following common characteristics:

(a). **Robust NAS networks’ cells are heavily populated with multiple branch network structures.** Multiple branch network structures are prevalent in typical robust NAS networks shown in Table I. In addition, the structure of multiple branches accounts for more than 35% of the entire network model, while the corresponding weights are 61% on average. More importantly, each branch might have different execution times and varying number of layers.

(b). **Robust NAS networks have both backward and forward dataflows between parallel branches, while typical DNN networks only have forward dataflows.** As shown in

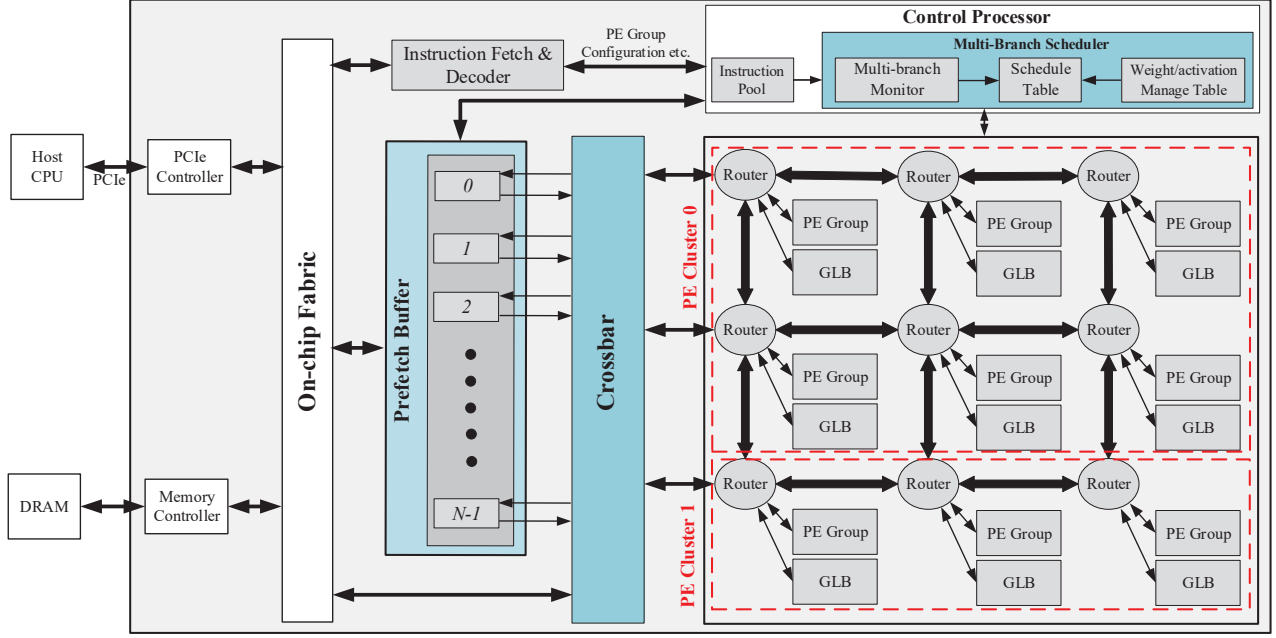


Fig. 2: The architecture of NASGuard accelerator.

Figure 1(b), multiple branches belonging to different convolution kernels inside the cell can have data communications. We have observed that the percentages of branches with external communication and backward dataflows in the computational graph are 20% ~ 35%. These multi-dataflow structures usually converge after learning different feature representations. In particular, these high-dimensional features are generally connected by *concat* or *element-wise add* operations, and the latter is utilized more frequently in real applications.

(c). Vast shortcut/dense connections exist intra/inter-cell of robust NAS networks. Shortcut [35] and dense connections [42] are introduced to address the vanishing gradient problem in deep network structures, which is critical in developing highly accurate DNNs [35], [42]–[44]. Such connections are used to pass feature data downward to achieve deeper networks. They contain *long-term data dependency* and are prevalent between cells/layers. There are also data communications across cells that have *multi-input/output nodes* as shown in Figure 1. These irregular dataflows and data dependencies require frequent accesses to the off-chip memory, which reduces efficiency of existing DNN accelerators.

In addition, retraining typical DNN networks with adversarial training methods also resist adversarial example attacks, which results in complex dataflows and diverse network structures [35], [42]–[44]. All of these network architectures pose various challenges to existing AI systems. Therefore, it is vital to develop a unified DNN accelerator architecture that can support the characteristics of various networks, especially robust NAS networks. The key design objective of the corresponding accelerator architecture is to maximize data reuse and resource utilization for multi-branch parallel execution, so that they can

be efficiently deployed to perform inference prediction. This is a main objective of NASGuard.

B. Problems of Deploying Robust NAS Network on Existing DNN Accelerator Architectures

Current DNN accelerators usually adopt monolithic systolic array architecture to execute each DNN layer serially. They usually contain a two-dimensional PE array and hierarchical on-chip buffers [38], [39], [41], [45]. The systolic array architecture, which is represented by Google TPU [37], **cannot efficiently tap into the parallelism of robust NAS networks**. The polygonal line in Figure 3 shows the computation to communication ratio (CCR) for various operations. These operations have different communication patterns and exacerbate the DNN accelerator’s underutilization problem. Moreover, current DNN accelerators (e.g., TPU) can only execute each branch within the cell serially¹, which can cause long periods of severe underutilization of some resources. As a matter of fact, the average utilization rate of PEs is only 30% ~ 64.5%.

Although multi-core architecture designs represented by Dadiannao [46] can support multi-branch parallel computing [47], [48], their coarse-grained task scheduling results in **low resource utilization**. As shown in Figure 3, different network operations have different PE utilization and bandwidth utilization rates. Therefore, different branches composed of multiple network operations will be burdened by the branch with the lowest performance. Although partial feature maps

¹Layer fusion technique can reduce memory traffic and improve performance, but it also brings computation redundancy and increases the demand for on-chip buffer capacity. NASGuard supports a layer fusion technique for robust NAS networks under limited on-chip buffer capacity.

can be reused in data communication within the multi-core, it is ineffective for multi-branch dataflows and communication, and fails to minimize the off-chip memory traffic.

The many-core architecture represented by Eyeriss-v2 [49] can support simultaneous execution of robust NAS networks' multiple branches at a finer granularity, but the **data locality** of multiple branches between cells or within a cell is not fully exploited due to (1) lack of efficient mapping designs for computing and communication resources, (2) lack of high-bandwidth access support for memory-intensive branches, (3) inability to fully exploit intra/inter-branch dependencies, and (4) inability to reuse data from previous layers (e.g., shortcut/dense connection).

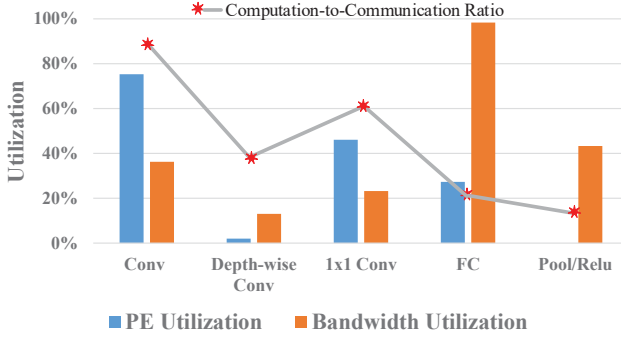


Fig. 3: PEs and bandwidth utilization when running typical robust NAS networks.

Aiming at the aforementioned problems, we proposed to optimize the computational efficiency of robust NAS networks from three computational perspectives: (1) mapping computational branches to many-core architectures for better data locality and improved resource allocation; (2) applying multi-grained dynamic resource scheduling to make full use of hardware resources; and (3) optimizing data prefetch for reducing the idle time of PE array.

III. ARCHITECTURE DESIGNS

The design goal of NASGuard is to fully exploit the accelerator's computing resources and improve data locality by (1) creating a heuristic-based multi-branch resource mapping to NASGuard, (2) designing a multi-granularity scheduling mechanism (layer-level and tile-level) for load balancing of computation tasks, and (3) implementing a hardware prefetcher directed by the compiler.

A. NASGuard Overview

The overall architecture of NASGuard is depicted in Figure 2. It contains the following major components.

Computing resources To support the parallel execution of multiple branches, computation of each branch is mapped to a PE cluster in Figure 2, the cluster consists of multiple PE groups (PG), where each PG is equipped with a GLB. NASGuard not only supports parallel execution of branches, but also dynamically allocates resources for workload balance at runtime. Specifically, if the execution time of different

branches are significantly different, branches with shorter execution time will release some of resources and give them to branches with longer execution time. If the execution time of different branches are not much different, no branches will release any resources, but branches with shorter execution time will steal data from branches with longer execution time. When doing data stealing, the scheduler uses proper status registers to guarantee each piece of datum to be processed by only one branch.

Memory hierarchy NASGuard has a three-level memory hierarchy. First, the private buffer in each PG provides input data for the PE array and stores the computing results. Second, a GLB is paired with each PG and records weights and input/output activation values from the off-chip memory or prefetch buffer. Third, a prefetch buffer in Section III-D serves as the last level memory hierarchy between the PGs and off-chip memory to fetch weights and activation values early.

Control processor is mainly used to distribute and schedule commands. Its key component is the multi-branch scheduler (in Section III-C). It is mainly responsible for three jobs: (1) the allocation of computational resources for multi-branch mapping; (2) the dynamical scheduling of branches for load balancing; (3) the execution of data prefetching operations based on the performance prediction model.

Putting it all together, NASGuard provides not only efficient parallel computing power for multiple branch networks within a cell, but also effective data prefetching for shortcut/dense connections.

B. PE Group Microarchitecture

Figure 4 illustrates the PG microarchitecture. As depicted, the PG comprises a set of PEs. The PEs and private buffers within the PG provide computing capabilities for convolution and inner product layers while customizable normalization, pooling, and activation modules in nonlinear unit support for other kinds of layers. These features allow users to generate a concrete accelerator with any number of PGs and PEs-per-PG. Neighboring PGs have a unidirectional link that forwards input data from a PG with lower index (PG_{i-1}) to the adjacent PG at the higher index (PG_i).

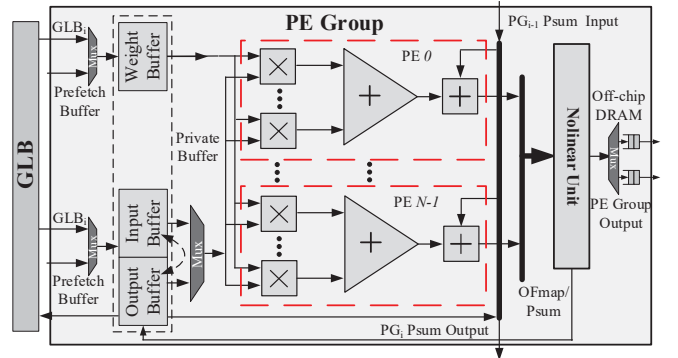


Fig. 4: The microarchitecture of the PG.

Private buffers The private buffers fetch weights and input data from local GLB and also obtain prefetched weights and activation values from the prefetch buffer. They can be split into three types of buffers. The input neurons buffer (NBin) and the output neurons buffer (NBout) record the intermediate feature maps. The data in the NBin/NBout can be logically swapped for efficient data transfers. The synaptic weights buffer (SB) which stores weights are shared across all the PE units within each PG.

GLBs GLB can be implemented as either a central shared buffer or multiple physically distributed local buffers. NASGuard uses the distributed design to support concurrent execution of multiple branches on PGs. Each local GLB can also be logically divided into three sub-buffers with separate ports, namely, input activation buffer, output activation buffer, and weight buffer.

Special data channel for element-wise add The *element-wise add* performs the addition calculation using the corresponding data channels of multiple nodes as inputs. In general, one typical DNN accelerator first needs to write the channel data of each node into the off-chip memory. After the data of the last channel of the last node is calculated, the data of its previous channel is reloaded back as the source operands. Since the memory access patterns of robust NAS networks can be known at compile time, the compiler can direct the hardware to properly forward data after the last-level adders. To support that, NASGuard has a designated special data channel for the last-level adders (additional adders are introduced in [50]) that avoids off-chip DRAM accesses.

In order to make the correct add computation of the corresponding data channels in each node, it is imperative to have a synchronization mechanism for these concurrent operations. However, it is heavy-weight to adopt a full-fledged coherence protocol or a lock-based synchronization scheme. Thus, we choose the first-come-first-served (FCFS) policy to conduct *element-wise add* operation in each channel. Noted that the intermediate data unused at the current or next level can still be written directly to the off-chip memory.

C. Multi-branch Scheduler

The multi-branch scheduler has three key components: scheduling tables (e.g., branch scheduling tables, load balancing scheduling tables, etc.), weight/activation management tables and a branch monitor. It is invoked whenever (1) a new branch task is dispatched, (2) a fine-grained schedule is performed, or (3) a hardware prefetch operation is triggered. It supports the following functions.

Allocate resources for parallel execution of branches NASGuard initializes the branch scheduling tables with the hardware mapping information generated by the heuristic multi-branch mapping model in Section V. According to the hardware configuration information from the branch scheduling tables, the scheduler configures the NoC routing tables for PE cluster and GLB partitioning.

Implement the two-level scheduling mechanism On our hardware implementation, each scheduling table corresponds

to a branch, and each entry of the scheduling table corresponds to one layer within the branch. Each scheduling table entry contains layer-level/tile-level scheduling information (such as layer/tile index, layer/tile counter, and layer/tile dependencies, etc.) that is used for fine-grained scheduling. These tables are initialized with the statically determined information generated by the compiler.

Monitor and trigger hardware prefetch operations The weight/activation management tables are used to track the address changes of the weights and activation values for each branch. The branch monitor sends the execution status of a given branch or node to the multi-branch scheduler. When the scheduler detects that the execution of the given branch or node approaches a prefetching point specified by the compiler, a corresponding prefetching operation is triggered.

D. Prefetch Buffer Microarchitecture

We evaluate the multi-level memory architecture using the test environment in Section VIII-A. Compared to the Level 1 memory hierarchy, the results show that Level 3 memory hierarchy contributes to the greatest power reduction ($3.7\times$) and the maximum data reuse (63.1%) for robust NAS networks and supports all degrees of temporal and spatial localization in the data access patterns. More importantly, utilizing the Level 3 on-chip memory structure as a prefetch buffer is a natural choice, as it is able to cooperate with the multi-branch scheduler to prefetch data timely.

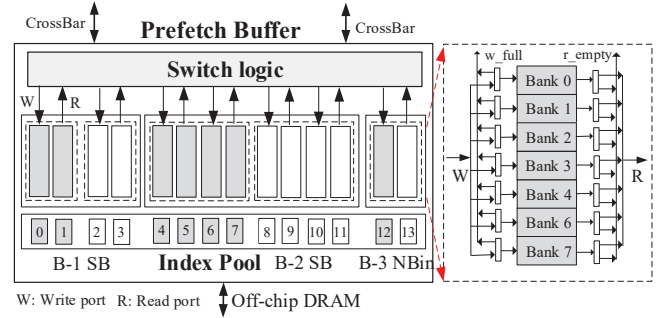


Fig. 5: The multi-bank prefetch buffer.

Prefetch buffer with multi-bank hierarchy As shown in Figure 5, each bank has a unique physical index and dedicated access state bits (w_full and r_empty). Based on these information, a hardware list is maintained for bank management. When a branch's data in one prefetch buffer is used, the prefetch buffer is set to the inactive state, and the corresponding bank index is recycled. Doing so improves the utilization of the on-chip prefetch buffer. Each logical buffer is designed with a double buffer to hide latency and maintain higher bandwidth.

Activation buffer reuse The lifespans of feature tensors of multiple layers are usually different, which provides an opportunity of buffer reuse. A global liveness analysis method in [51] can be applied to the computation graph of a given DNN model for buffer reuse. NASGuard contains parallel PE

clusters, and each PE cluster needs to access its weights and activation values. As shown in Figure 5, the entire SRAM is divided into N logical buffers to enable concurrent accesses by N parallel PE clusters. NASGuard achieves fine-grained reuse of the activation buffer via such a flexible buffer management mechanism.

IV. NASGUARD COMPILER

Robust NAS networks have stable execution behaviors. For example, the order of memory accesses is deterministic, and the execution time is predictable. These characteristics make the compiler more efficient in resource scheduling and hardware/software collaboration. Figure 6 illustrates the overall workflow of NASGuard. First, the compiler takes the specification of robust NAS network as input ① and parses them into computational graphs. Second, the heuristic multi-branch mapping model and topology-aware performance prediction model ② take the computational graphs of networks as inputs, and generate configuration tables ③ for each branch within the cell. The off-line configuration tables store all relevant information of robust NAS networks, such as hardware mapping information, scheduling information, and weight/activation management information. Third, loop oriented optimizations of the compiler backend produces executable binaries ④. Last, the runtime environment on host CPU uses the configuration tables to initialize the scheduling tables ⑤ and the weight/activation management tables within the multi-branch scheduler to perform tasks scheduling and resource allocation.

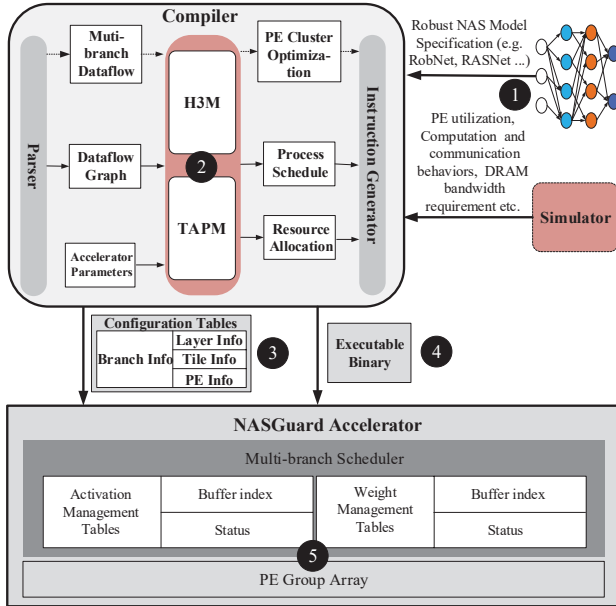


Fig. 6: Overall workflow of NASGuard.

The heuristic multi-branch mapping model (H3M) is responsible for allocating computing and storage resources for multi-branch parallel execution. The topology-aware performance prediction model (TAPM) produces the prediction

information for nodes and branches of robust NAS networks at compile time. It coordinates with the multi-branch scheduler for data prefetch operations at runtime. Two models alleviate manual effort to derive the optimal parameter configurations and achieve better data locality and parallelization. Experimental results demonstrate that NASGuard can be utilized near the peak hardware capability and achieve a computational efficiency of over 96.74% on typical robust NAS networks in Section VIII-B.

V. MULTI-BRANCH MAPPING MECHANISM TO ALLOCATE RESOURCE ON DEMAND

In order to exploit computational parallelism and data locality, the multi-branch mapping is founded upon four principles.

- Each branch should be allocated a reasonable amount of computing resources according to its needs.
- It is recommended to map branches with similar execution time onto the nearby PE clusters for simultaneous execution and efficient inter-branch scheduling.
- Branches with data dependencies and frequent communications are mapped onto the PE clusters located in the same region to reduce the inter-clusters traffic.
- A branch with backward dataflows should be located close to its consumer branch, so that branches with dependencies can build a pipeline for data reuse and thus reduce potential data congestion.

The heuristic multi-branch mapping model that conforms with these principles is proposed in Figure 6, and is used to generate hardware mapping information of each branch. As shown in Section III-C, the branch scheduling table is initialized based on the hardware mapping information generated by the following three steps.

- Each branch is extracted into a directed acyclic graph (DAG) by the compiler for analyzing the relationship and dependencies between various operators. In addition, some of the key information about each branch, including computing intensity, memory intensity, resource utilization and execution time, are obtained from the simulator.
- Based on the above data, the Agglomerative Hierarchical Clustering mechanism (AHC) [52] is adopted to classify each branch. AHC technique is used to classify a set of objects into distinct clusters so that objects sharing similar properties are clustered together.
- Taking full advantage of “Cannikin Law” caused by the difference in execution times of multiple branches, the Shortest Job First (SJF) strategy is adopted to fine-tune the clustering results into a hardware mapping table, which is also dependency-aware and statically scheduled.

The hardware mapping table records the computation and storage configuration for each PE cluster. It is used to initialize the scheduling tables cell by cell rather than layer by layer. In addition, the attached table which contains resource scheduling information (e.g., two PE cluster’s resources are merged) is used to dynamically reallocate resources within the cell.

VI. MULTI-GRANULARITY SCHEDULING MECHANISM FOR LOAD BALANCING

A. Inter-branch Layer-level Scheduling

Because different branches might have different execution behaviors, the parallel execution of multiple branches has to face the issue of load imbalance. To improve memory bandwidth and resource utilization of PE clusters, we design an inter-branch scheduling algorithm at the layer level. The algorithm contains two policies. 1) Before allocating resources to the branch with the longest execution time, the computation of the current layer must have been completed. 2) When the execution time of the longest branch detected by the branch monitor is similar to that of the predicted value (an empirical margin has been set for this purpose), the original configuration is kept as unchanged and no more resource is allocated.

B. Intra-branch Tile-level Scheduling

As feature maps are usually larger than the size of the on-chip buffer, tiling technique is usually used [53], [54] to provide an opportunity to accelerate other tiles during the execution of the longest branch. A natural idea is to use the workload-steal strategy for fine-grained scheduling [55]. In particular, rather than releasing the computing resources after the execution of a short branch, a PE cluster is allowed to “steal” weights and input feature map tiles from the last unprocessed tile in another PE cluster for better resource utilization.

When the monitor detects that the difference between the real execution time and the predicted one is less than the predefined threshold, the workload-steal strategy will be triggered. Specially, the scheduler dispatches the instructions of the last unprocessed tile of the longest branch to an idle PE cluster, while the current PE cluster only needs to modify the destination addresses of the tile execution instructions. Instead of implementing the work-steal scheduler in hardware, we use a branch monitor with the corresponding scheduling parameters generated by the compiler to dynamically detect workload imbalance in multi-branch execution. The scheduler also needs to configure the crossbar properly so that the weights and feature map tiles are fetched from the slowest PE cluster to the idle PE cluster.

VII. PERFORMANCE PREDICTION MODEL FOR DATA PREFETCH

The multi-branch scheduler prefetches weights and activation values of the branch jump according to the branch scheduling tables and the weight/activation management tables. Meanwhile, the branch monitor is used to detect the execution status of the branch jump.

A. Opportunity: the Inference Performance is Predictable

A robust NAS network is composed of cells with multiple branches as a computational unit, and the structure of the network is fixed during the inference computation. The memory layout of the weight variants are known at compile time, and

the computation and memory behaviors of these layers are highly deterministic. These properties allow us to predict the branch execution order, and to know the data dependencies for issue prefetching operations in advance. To verify their predictability during inference, the following experiments have been conducted:

- With the same hardware configuration, we have evaluated the RTL implementation of NVDLA [41] and its performance model using typical DNN networks (AlexNet [33], GoogleNet [34], ResNet50 [35]). The results show that there is 0.91% ~ 1.63% deviation. It is mainly due to the difference between the theoretical PE/bandwidth utilization rates and the real values.
- We have validated the modified Maestro’s performance model [56] against our Verilog simulation of NASGuard with the same network type and hardware configuration. The results show that the execution time estimated by modified MAESTRO is on average 4.3% away from the results of the cycle-accurate Verilog simulation.

B. Topology-aware Performance Prediction Model (TAPM)

Due to the predictability of the inference process, we adopt maestro’s performance analysis model as our topology-aware performance prediction model. TAPM is used to evaluate the performance and memory access patterns of each branch. In order to accurately estimate the end-to-end latency of complex network, TAPM is designed to predict 1) the node-level execution time, 2) the number of nodes executed sequentially within a branch and the number of branches executed in parallel within a cell, and 3) the branch-level and cell-level overall execution time. TAPM takes the computation graphs of the network and the hardware configuration (e.g., the number of PG and GLB) of each branch as inputs, and generates the predicted execution time of each branch, each node within the branch, and the whole cell. It schedules computing-intensive branches with priority over memory-intensive branches. By analyzing time relations between the top and bottom layers using the TAPM, data access in the next layer can be prefetched in advance. Using the prefetching technique is an effective way to improve the utilization of computing and memory bandwidth. It also reduces the configuration latency by 36% compared to that without prefetching.

C. Prefetching Weights and Activation Values

The performance of each node and branch is predicted by TAPM in the compiler, and the prediction information is written into the scheduling tables in the multi-branch scheduler. The multi-branch scheduler manages the execution of branches with the same number of rows as the branch’s network layer, and contains key information such as the operation type, execution time, dependent relations, and etc. During the inference process, the branch monitor dynamically monitors the execution status of each branch. Once the monitored execution time is over the threshold, the multi-branch scheduler issues a prefetch operation to load the corresponding weights and activation values.

Weights prefetch NASGuard uses information from the branch scheduling table to predict *branch jump* and *backward branch* direction and perform weight prefetches. Hardware counters are applied to update the cross-layer dependencies in the branch scheduling tables. A one-dimensional array records the layer’s dependency parameter which may depend on two or three branches of the previous layer. As dependencies indicate jumps between branches or cells, NASGuard uses dependencies and layer indexes to track dependent relations between the upper and lower layer. When a layer’s execution is finished, the hardware counter of the corresponding dependency will minus one. When the counter of dependence becomes 0, a branch jumps to perform a weight prefetch operation.

The *weight management table* is designed to guide weight prefetch by implementing a free list like the prefetch buffer index pool to cache the weights of multiple branches. A status register is set in the weight prefetch buffer, including the processing status (*on-the-fly request*, *completed request*) and related data access status. They will be updated synchronously with the weight management table.

Activation values prefetch NASGuard uses a multi-branch scheduler to prefetch the activation values of the previous layer in advance (*e.g.*, *shortcut/dense connected layer*). Activation value prefetching adopts the same method as weight prefetching. The *activation management tables* also manage activation data and corresponding prefetch buffers. The buffer size for each branch is assigned and initialized at compile time. However, it can be partitioned dynamically during the execution of branches in the same cell.

VIII. EVALUATION

A. Methodology

We have implemented NASGuard using Verilog RTL. Table III lists the major architectural parameters of the evaluated platforms. To make a fair comparison, a typical many-core architecture without a multi-branch scheduler and a prefetch buffer is used as the baseline.

The Synopsys Design Compiler (L-2016.03-SP5) with SMIC 65nm technology is used to synthesize the designs for measuring their timing and area cost. And the power consumption is estimated with Synopsys PrimeTime PX according to the simulated value change dump file. In order to assess its energy consumption for the off-chip memory access, a DDR4 memory model is integrated. The bandwidth is 16 GB/sec, and the average energy consumption of data access is 15 pJ/bit.

Simulation methodology We developed a cycle-accurate simulator based on the Maestro [56], which is an open-source simulator with flexible simulation framework of DNN accelerator. It simulates kinds of dataflows and on-chip memory hierarchy. Based on its NoC fabric, we extended the single PE to PG with a set of scheduling registers inside, and added the logic of one multi-branch scheduler in the control processor. Besides, we also enhanced the L3 cache to support prefetch buffer. In addition, the TAPM prediction model has been cross-validated with both SCALE-Sim [60] and Maestro to tune its

TABLE II: Networks selected for evaluation

Network	Type	Total Weights	Total MACs	Layer Operation
RobNet-small [11]	RNAS	4.41M	538M	Cell, Block, Dense connect, DWconv etc.
RobNet-large [11]	RNAS	6.89M	921M	Cell, Block, Dense connect, DWconv etc.
RASNet [12]	RNAS	7.42M	917M	Block, Conv2d, Shortcut connect etc.
RobustDARTS [57]	RNAS	4.7M	574M	Cell, Block, Conv2d, Shortcut connect etc.
MnasNet [58]	NAS	4.38M	340M	Block, Shortcut connect, DWconv etc.
NASNet-large [18]	NAS	88.76M	47.8G	Cell, Shortcut connect, DWconv, Conv2d etc.
DenseNet121 [42]	Hand	7.98M	2.9G	Dense connect, Conv2d.
Inception-V3 [59]	Hand	27.16M	5.75G	Inception module, Conv2d, Pool etc.
ResNet50 [35]	Hand	25.56M	4.14G	Shortcut connect, Conv2d etc.

TABLE III: Key design parameters and area breakdown.

	Architectural Components	Baseline		NASGuard	
		Size or number	Area (mm ²)	Size or number	Area (mm ²)
PG	PEs per PG	16	0.112	16	0.112
	Mux	N/A	0	3	0.003
	Partial Sum Register	N/A	0	48B	0.004
	IO FIFOs	N/A	0	144B	0.011
	Weight Buffer	0.5KB	0.012	0.5KB	0.012
	Input/Output Buffer	1KB	0.026	1KB	0.026
	Control Logic	N/A	0.012	N/A	0.021
GLB	Total Buffer	1.5MB	26.88	768KB	13.43
Accel*	PGs	64	10.37	64	12.09
	GLB	64	26.88	64	13.43
	Router	64	0.96	64	0.96
	Control Processor	N/A	1.43	–	1.28
	Branch Scheduler	N/A	0	–	1.12
	Prefetch Buffer	N/A	0	768KB	13.71
Total Area		39.64 mm ²		42.59 mm ²	

*represents accelerator.

accuracy. And we used Verilog implementation to adjust the accuracy of the simulator for accurate profiling functions.

Workloads Table II introduces the evaluated models including typical hand-designed DNNs (hand), NAS networks (NAS) and robust NAS networks (RNAS). They are from various applications ranging from image classification, segmentation, and object recognition. And those popular networks with medium to large scale have several hundreds of Mbytes memory footprints. Moreover, they share some common structural features of the robust NAS networks.

B. Experimental Results

(1) Performance speedup, area and power overhead

a) Overall performance speedup and power reduction

Figure 7 illustrates the experimental results compared with baseline architecture. NASGuard achieves $1.74\times$ speedup and $1.26\times$ power reduction over baseline on average. Since RobNet-large has all the typical characteristics of a robust NAS network, the highest speedup ($2.13\times$) is observed. Another case is MnasNet, which has a higher proportion of depth-wise convolution. This makes PE less utilizable and does not benefit from NASGuard's optimization techniques, such as work-steal strategies. As a result, the performance speedup of NASGuard is modest for MnasNet network.

It is important to note that NASGuard can also effectively support a wide variety of DNN networks. This is also one of NASGuard's strengths over the design of a monolithic systolic array. Since these typical hand-designed DNN networks usually have shortcut/dense connections and inception modules, these features could be fully exploited by NASGuard architecture.

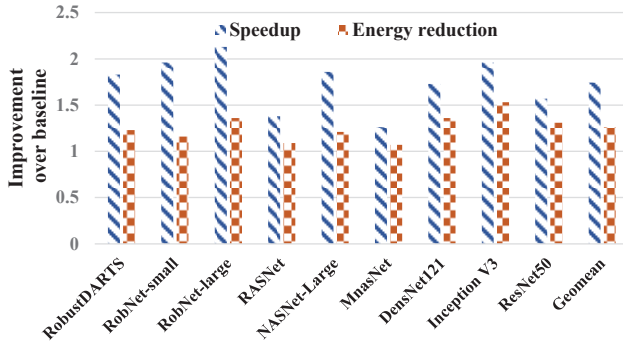


Fig. 7: NASGuard performance and energy relative to the baseline.

As shown in Figure 7, NASGuard saves 7% ~ 36% power consumption over baseline architecture ($1.26\times$ reduction on average). Power saving comes from two factors. First, the execution time is reduced due to higher utilization of PE and bandwidth. Thus, both static and dynamic power is saved. Second, NASGuard maximizes data reuse by performing pipeline computation on dependent layers within the cell, and special data channel further avoids unnecessary off-chip memory access. They effectively reduce the dynamic power consumption. Although additional logics in NASGuard definitely increase the static power, the above analysis explains the overall power reduction.

b) Area overhead and breakdown

Table III summarizes the area breakdown of the major microarchitectural components. Overall, the area overhead of NASGuard is increased by 7.4% in comparison with baseline. The area breakdown in Figure 8(a) shows that the modular of PGs and GLB dominate the area overhead, while the area of the branch scheduler only accounts for 2.6% of the total area. Within each PG, all of

the on-chip buffers occupy about 24.9%, while the PE array makes up 59.3% in Figure 8(b).

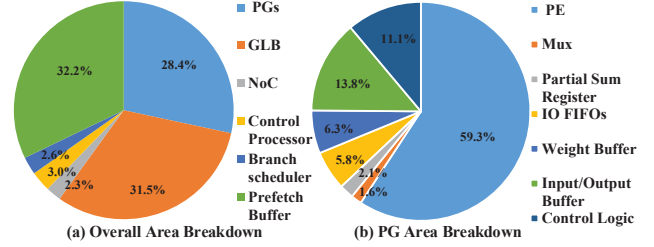


Fig. 8: Silicon area breakdown.

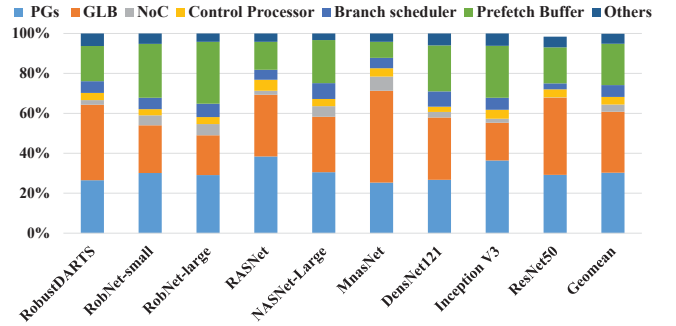


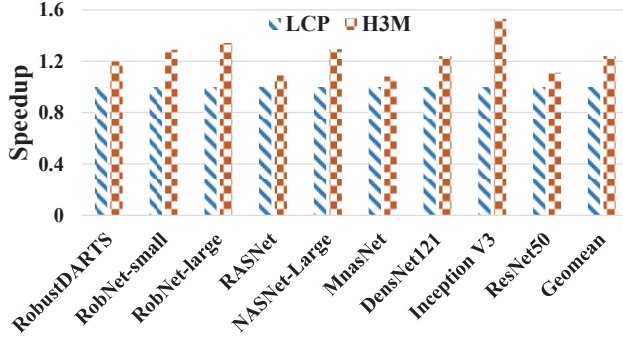
Fig. 9: Power breakdown for different networks.

c) Power breakdown Figure 9 illustrates the power breakdown of NASGuard running a variety of robust NAS networks and DNNs. It can be observed that power breakdown is dependent on the characteristics of networks. In particular, when running RobNet-Large network, the power consumption of the NoC and logic prefetch buffer is higher. The reason is that the network induces more prefetching operations, and the workload imbalance induced by parallel execution of multiple PE clusters triggers more work-steal operations.

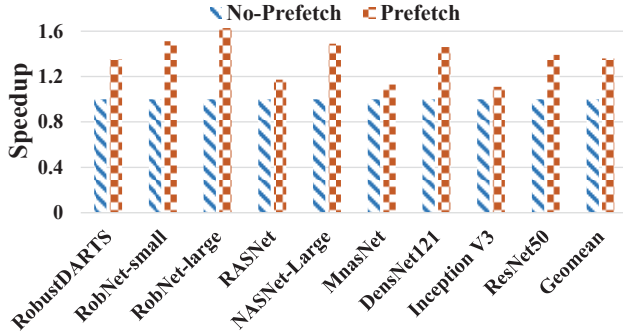
Compared with the baseline architecture, the static power consumption of the NASGuard is increased by 12.8%. This overhead is mainly from the multi-branch scheduler and prefetch buffer. In particular, the former results in an 8.1% overhead increment, while the rest increment is contributed by the latter and related control logics.

(2) Performance evaluation for key designs

a) Evaluation on multi-branch mapping model The Layer Clusters Paralleling mapping method (LCP) [61] for accelerating inception networks (multi-branch structure) can be applied to many-core accelerator architecture. In order to verify the effectiveness of our proposed heuristic multi-branch mapping model (H3M), LCP has also been conducted for comparison. As shown in Figure 10(a), NASGuard with H3M achieves an average performance improvement of $1.24\times$ than LCP. And the inception-V3 achieves the highest performance speedup ($1.53\times$). While LCP aims at optimizing sequential dataflow with the inception module, the proposed H3M optimizes the inception module for parallel execution, and reasonably allocates resources for each branch. Moreover, it builds a pipeline



(a) Comparison of two mapping strategies.



(b) Performance improvement of prefetch.

Fig. 10: Evaluation on key designs

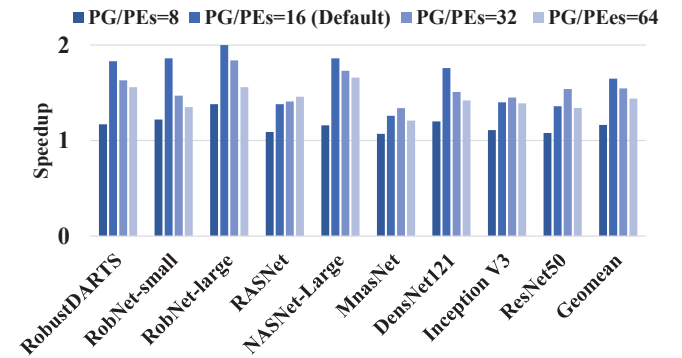
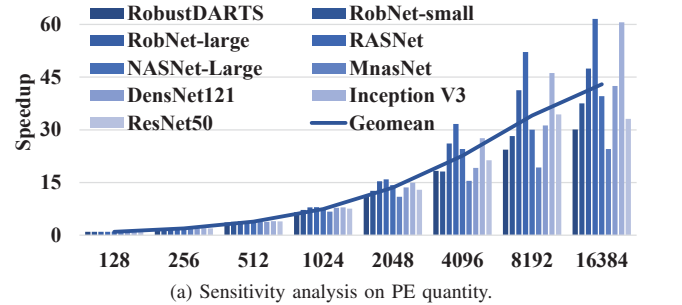
for the dependent branches of robust NAS networks, thus reducing the off-chip memory access and the NoC congestion of internal data communication. While LCP can maximize the reuse of input data and improve resource utilization by clustering operations of similar computing, it cannot optimize the complex network branches or the diversified computing operations inside cell.

b) Evaluation on performance prediction model and prefetch mechanism Three major scenarios are designed for TAPM evaluation, including early prediction, timely prediction, and lazy prediction. Overall, TAPM effectively predicts the execution time of cell node and branch, and the false prediction is 1.04%. In essence, the effectiveness of TAPM lies in its ability to reduce the waiting time of the PE array. The result shows that such ability is closely related with the policy of either early or timely prediction rather than the lazy one.

Figure 10(b) compares the performance when enable and disable the prefetch functionality. The results show that NASGuard with prefetch witnesses an average 35.9% performance improvement than the case without prefetch (non-prefetching). First, in case of RobNet-large and DesNet121, the performance speedup is 62% and 46% respectively. Weights and input feature maps prefetched in these two cases account for 84.7% and 61.3% of their total input data. Structural characteristics of these networks provide the opportunities of exploiting the data locality at the level of cells and branches. And the

prefetching technique also reduces memory bandwidth and thus improves resource utilization. Second, for the memory-intensive networks which have poor memory locality (RASNet), the performance improvement of enabling prefetch is limited (about 12%). Third, in case of the computing-intensive networks (e.g., MnasNet which has multiple DWconv [58], [62]), the multi-branch scheduler also has fewer opportunities to dispatch prefetching operations.

c) Evaluation on the combination of prefetch and load balancing scheduling It is obvious that the baseline architecture severely suffers from the issues of the irregularity of memory accesses and load imbalance. The performance of baseline is only 47.35% ~ 60.54% of NASGuard design. By employing prefetching and multi-granularity scheduling mechanisms, NASGuard can effectively address the workload imbalance and achieve high utilization (79.29% ~ 96.74%) of computational resources. Since MnasNet has many computing-intensive DWconv layers, it can reduce the opportunity to overlap prefetching of activation values, it thus achieves limited performance improvement (1.26 $\times$). And other robust NAS networks achieve higher performance improvements by means of prefetching and multi-granularity scheduling mechanism.



(b) Sensitivity analysis on the number of PEs per PG.

Fig. 11: Sensitivity analysis.

(3) Scalability

Figure 11(a) shows the sensitivity analysis on the PE quantity. For all the cases, the PG always has 4x4 PEs. And we select the case of 128 PEs as the baseline. It can be observed that the performance improvement scales almost linearly from 128 to 2048 PEs. MnasNet, a DWconv-dominated network, achieves the lowest of the linearly scaled performance, 23.5%,

at 16384 PEs. The number of PE increases from 2048 to 16384, but the computational performance does not increase linearly and PE utilization decreases significantly with the increase of PE number. This is because DWconv has low parallelism and poor data locality. Another reason is that the performance is also dependent on the latency of NoC communication.

(4) Sensitivity analysis of the number of PE per PG

The many-core architecture exposes two levels of parallelism: The first is the PE-level parallelism in a PG. It is a fine-grain one which offers plenty of adjacent output elements. And the second is the PG-level parallelism. It is a coarse-grain one which usually generates many independent output feature maps. Figure 11(b) evaluates the sensitivity of the number of PE per PG. The number of PEs is varied from 8 to 16, 32, and 64 within the PG. It can be found that employing a small number of PEs (8) is not a good choice. The main reason is that the inter-PE fragmentation problem aggravates with limited parallel multipliers residing in a PG. The PG with a higher number of PEs indicates more opportunities for parallelism, which is more beneficial for utilization improvement. However, as the number of PEs gets larger (32 or 64), the limited performance improvement is observed.

Due to the different algorithmic properties of the load network, the network operations within multiple branches have different parallelism. And some layers inherently perform better if they are tiled to smaller chunks and parallelized across multiple smaller arrays, because in that case they would exploit coarse-grain parallelism and yield better resource utilization. In short, the number of PE needs to be carefully decided according to the characteristics of the load network. Fortunately, our architecture can flexibly form PE clusters of different granularity for each branch and achieve the requirements of multi-branch parallelism.

IX. DISCUSSION

NASGuard can be broadened for deploying general DNNs Since the majority of early DNN networks usually have single-branch structure and contain a lot of parallelism, monolithic systolic arrays such as TPU are able to fully exploit their parallelism. As a result, NASGuard architecture which adopts many-core design might cause performance loss. It is important to note that the latest evolution of DNNs is the introduction of special structures such as multi-branch structure, depth-wise convolution, and shortcut/dense connections, etc. The design of large PE arrays cannot effectively exploit such kind of branch-level parallelism. In addition, the depth-wise convolution operation can further reduce PE utilization. It should be noted that NASGuard can effectively meet such a new trend. Its many-core design and data prefetch capability fully exploit the branch-level parallelism and maintain high PE utilization at every layer of the network. We select two typical recent DNNs (DenseNet121 [42] and Inception-V3 [59]) for this evaluation. Compared to Eyeriss-v2 [49] with the same area, the average performance speedup is $1.41 \times$ and

the energy is reduced by 36%. Such evaluation results validate our aforementioned hypothesis about NASGuard's efficiency for state-of-the-art DNN networks.

Differences between NASGuard and multi-branch neural networks optimization techniques System-level or software optimization techniques are usually used to accelerate the inference and scheduling of multi-branch neural networks [63]–[66]. They apply computation graph splitting and fusion techniques to perform serial execution on DNN accelerators. However, NASGuard adopts the design of the multi-branch scheduler to manage task scheduling for efficiently parallel execution of multiple branches. Moreover, software/hardware co-design enables NASGuard to reuse and prefetch data for various network structures (e.g., shortcut or dense connections).

X. RELATED WORK

This paper offers robust NAS network acceleration through architecture improvements that are propelled by unique computational characteristics and parallelism mechanisms of robust NAS networks. Moreover, NASGuard lies at the intersection of AI algorithms, security, and DNN acceleration. The relevant related work categories are discussed as below.

DNN accelerators Existing DNN accelerators [40], [67]–[71] have a fixed-sized systolic array with a number of PEs, and they are well matched for large and compute-bound DNNs; whereas, computation- and memory-bound robust NAS networks suffer from PE underutilization and fail to achieve high performance and energy efficiency [39], [45], [49], [54], [69], [72]–[76]. Although DNNGuard [77] supports most adversarial example detection methods, it does not effectively support robust NAS networks with multiple branches, and serially executes multiple branches within the cell, and does not fully exploit the internal data locality. For the same workload, our architecture has an average performance improvement of $2.18 \times$ compared with DNNGuard. It reduces data reuse between cells and PE utilization of single branch execution. In contrast to the monolithic systolic array, this paper explores a many-core architecture design for spatial collocation of robust NAS networks for multi-branch parallel execution and scheduling challenges. DeepSniffer [78] proposes the first quantitative systematic schema to explore the DNN model leakage risks through the DNN system, and Ptolemy [79] designs a hardware architecture support for robust deep learning against adversarial example. These works demonstrate that the designs of hardware architecture to resist adversarial example attacks is a promising research.

Hardware-aware NAS Experts with cross-disciplinary knowledge generally spend considerable amount of time and computing resources with hardware-aware NAS to obtain network models for dedicated hardware platforms [80]–[85]. Such an effort is challenged by the complexity and diversity of the hardware design space, as it is extremely difficult to find efficient networks corresponding to the various hardware architectures. Moreover, existing hardware designs do not cover the characteristics of NAS networks, resulting in

low utilization of hardware resources. NAS networks on our hardware architecture have higher accuracy and robustness, as well as better inference performance. The main objective of this paper is not to design the best DNN accelerator, but to provide hardware designers with a trade-off method to evaluate more comprehensive design options, and to design a tightly coupled accelerator that balances the performance, accuracy and robustness of DNN algorithms.

XI. CONCLUSION

The security of deep learning system is a critical problem, therefore, accelerating robust NAS networks is arguably as important as accelerating DNNs themselves. However, current DNN accelerator architectures are incapable or ineffective to run robust NAS networks. This paper proposes a concrete DNN accelerator architecture called NASGuard, which fills an important gap. It offers robust NAS network acceleration through architecture improvements that are motivated by unique computational characteristics and parallelism of this kind of new networks.

ACKNOWLEDGMENT

We thank the anonymous reviewers for their insightful comments and suggestions. This work was supported by the Strategic Priority Research Program of Chinese Academy of Sciences under grant No. XDC02010000. The corresponding author is Rui Hou (hourui@iie.ac.cn).

REFERENCES

- [1] T. Tsai, K. Yang, T.-Y. Ho, and Y. Jin, "Robust adversarial objects against deep learning models," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 01, 2020, pp. 954–962.
- [2] A. Athalye, N. Carlini, and D. Wagner, "Obfuscated gradients give a false sense of security: Circumventing defenses to adversarial examples," *arXiv preprint arXiv:1802.00420*, 2018.
- [3] H. Qiu, C. Xiao, L. Yang, X. Yan, H. Lee, and B. Li, "Semanticadv: Generating adversarial examples via attribute-conditional image editing," *arXiv preprint arXiv:1906.07927*, 2019.
- [4] J. Li, S. Ji, T. Du, B. Li, and T. Wang, "Textbugger: Generating adversarial text against real-world applications," *arXiv preprint arXiv:1812.05271*, 2018.
- [5] A. Kurakin, I. Goodfellow, and S. Bengio, "Adversarial examples in the physical world," *arXiv preprint arXiv:1607.02533*, 2016.
- [6] C. Yang, A. Kortylewski, C. Xie, Y. Cao, and A. Yuille, "Patchattack: A black-box texture-based attack with reinforcement learning," *arXiv preprint arXiv:2004.05682*, 2020.
- [7] Y. Zhong and W. Deng, "Towards transferable adversarial attack against deep face recognition," *arXiv preprint arXiv:2004.05790*, 2020.
- [8] Berkeley, "Physical adversarial examples against deep neural networks," <https://bair.berkeley.edu/blog/2017/12/30/yolo-attack/>, 2018.
- [9] D. Yang, C. Xiao, B. Li, J. Deng, and M. Liu, "Realistic adversarial examples in 3d meshes," *arXiv preprint arXiv:1810.05206*, vol. 2, 2018.
- [10] K. Xu, G. Zhang, S. Liu, Q. Fan, M. Sun, H. Chen, P.-Y. Chen, Y. Wang, and X. Lin, "Adversarial t-shirt! evading person detectors in a physical world," in *European Conference on Computer Vision*. Springer, 2020, pp. 665–681.
- [11] M. Guo, Y. Yang, R. Xu, Z. Liu, and D. Lin, "When nas meets robustness: In search of robust architectures against adversarial attacks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 631–640.
- [12] S. Kotlyan and D. V. Vargas, "Towards evolving robust neural architectures to defend from adversarial attacks," in *Proceedings of the 2020 Genetic and Evolutionary Computation Conference Companion*, 2020, pp. 135–136.
- [13] D. Yin, R. Kannan, and P. Bartlett, "Rademacher complexity for adversarially robust generalization," in *International Conference on Machine Learning*, 2019, pp. 7085–7094.
- [14] R. Gao, T. Cai, H. Li, C.-J. Hsieh, L. Wang, and J. D. Lee, "Convergence of adversarial training in overparametrized neural networks," in *Advances in Neural Information Processing Systems*, 2019, pp. 13 029–13 040.
- [15] M. Cheng, Q. Lei, P.-Y. Chen, I. Dhillon, and C.-J. Hsieh, "Cat: Customized adversarial training for improved robustness," *arXiv preprint arXiv:2002.06789*, 2020.
- [16] D. Wu, Y. Wang, and S.-t. Xia, "Revisiting loss landscape for adversarial robustness," *arXiv preprint arXiv:2004.05884*, 2020.
- [17] B. Li, S. Wang, S. Jana, and L. Carin, "Towards understanding fast adversarial training," *arXiv preprint arXiv:2006.03089*, 2020.
- [18] B. Zoph, V. Vasudevan, J. Shlens, and Q. V. Le, "Learning transferable architectures for scalable image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 8697–8710.
- [19] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," *arXiv preprint arXiv:1412.6572*, 2014.
- [20] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," *arXiv preprint arXiv:1706.06083*, 2017.
- [21] S.-M. Moosavi-Dezfooli, A. Fawzi, and P. Frossard, "Deepfool: a simple and accurate method to fool deep neural networks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 2574–2582.
- [22] J. Su, D. V. Vargas, and K. Sakurai, "One pixel attack for fooling deep neural networks," *IEEE Transactions on Evolutionary Computation*, vol. 23, no. 5, pp. 828–841, 2019.
- [23] S. Baluja and I. Fischer, "Adversarial transformation networks: Learning to generate adversarial examples," *arXiv preprint arXiv:1703.09387*, 2017.
- [24] X. Chen and C.-J. Hsieh, "Stabilizing differentiable architecture search via perturbation-based regularization," *arXiv preprint arXiv:2002.05283*, 2020.
- [25] H. Chen, B. Zhang, S. Xue, X. Gong, H. Liu, R. Ji, and D. Doermann, "Anti-bandit neural architecture search for model defense," *arXiv preprint arXiv:2008.00698*, 2020.
- [26] D. V. Vargas and S. Kotlyan, "Evolving robust neural architectures to defend from adversarial attacks," *arXiv preprint arXiv:1906.11667*, 2019.
- [27] C. Devaguptapu, D. Agarwal, G. Mittal, and V. N. Balasubramanian, "An empirical study on the robustness of nas based architectures," *arXiv preprint arXiv:2007.08428*, 2020.
- [28] J. Nandy, W. Hsu, and M. L. Lee, "Approximate manifold defense against multiple adversarial perturbations," *arXiv preprint arXiv:2004.02183*, 2020.
- [29] N. Carlini and D. Wagner, "Towards evaluating the robustness of neural networks," in *2017 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2017, pp. 39–57.
- [30] P.-Y. Chen, Y. Sharma, H. Zhang, J. Yi, and C.-J. Hsieh, "Ead: elastic-net attacks to deep neural networks via adversarial examples," *arXiv preprint arXiv:1709.04114*, 2017.
- [31] M. Dong, Y. Li, Y. Wang, and C. Xu, "Adversarially robust neural architectures," *arXiv preprint arXiv:2009.00902*, 2020.
- [32] Y. Dong, F. Liao, T. Pang, X. Hu, and J. Zhu, "Discovering adversarial examples with momentum," *CoRR abs/1710.06081*, 2017.
- [33] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in neural information processing systems*, 2012, pp. 1097–1105.
- [34] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2015, pp. 1–9.
- [35] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [36] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *arXiv preprint arXiv:1409.1556*, 2014.
- [37] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers et al., "In-datacenter performance analysis of a tensor processing unit," in *Proceedings of*

- the 44th Annual International Symposium on Computer Architecture, 2017, pp. 1–12.
- [38] T. Chen, Z. Du, N. Sun, J. Wang, C. Wu, Y. Chen, and O. Temam, “Diannao: A small-footprint high-throughput accelerator for ubiquitous machine-learning,” *ACM SIGARCH Computer Architecture News*, vol. 42, no. 1, pp. 269–284, 2014.
 - [39] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, “Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks,” *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, 2016.
 - [40] H. Kwon, A. Samajdar, and T. Krishna, “Maeri: Enabling flexible dataflow mapping over dnn accelerators via reconfigurable interconnects,” *ACM SIGPLAN Notices*, vol. 53, no. 2, pp. 461–475, 2018.
 - [41] NVIDIA, “Hardware architectural specification,” <http://nvidia.org/hw/v1/hwarch.html>, 2018.
 - [42] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, “Densely connected convolutional networks,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 4700–4708.
 - [43] S. Xie, R. Girshick, P. Dollár, Z. Tu, and K. He, “Aggregated residual transformations for deep neural networks,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 1492–1500.
 - [44] C. Szegedy, S. Ioffe, V. Vanhoucke, and A. Alemi, “Inception-v4, inception-resnet and the impact of residual connections on learning,” *arXiv preprint arXiv:1602.07261*, 2016.
 - [45] S. Venkataramani, A. Ranjan, S. Banerjee, D. Das, S. Avancha, A. Jagannathan, A. Durg, D. Nagaraj, B. Kaul, P. Dubey *et al.*, “Scaleddeep: A scalable compute architecture for learning and evaluating deep networks,” in *Proceedings of the 44th Annual International Symposium on Computer Architecture*, 2017, pp. 13–26.
 - [46] Y. Chen, T. Luo, S. Liu, S. Zhang, L. He, J. Wang, L. Li, T. Chen, Z. Xu, N. Sun *et al.*, “Dadiannao: A machine-learning supercomputer,” in *2014 47th Annual IEEE/ACM International Symposium on Microarchitecture*. IEEE, 2014, pp. 609–622.
 - [47] S. Lym and M. Erez, “Flexsa: Flexible systolic array architecture for efficient pruned dnn model training,” *arXiv preprint arXiv:2004.13027*, 2020.
 - [48] S. Yin, P. Ouyang, S. Tang, F. Tu, X. Li, S. Zheng, T. Lu, J. Gu, L. Liu, and S. Wei, “A high energy efficient reconfigurable hybrid neural network processor for deep learning applications,” *IEEE Journal of Solid-State Circuits*, vol. 53, no. 4, pp. 968–982, 2017.
 - [49] Y.-H. Chen, T.-J. Yang, J. Emer, and V. Sze, “Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices,” *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, vol. 9, no. 2, pp. 292–308, 2019.
 - [50] K. Hegde, R. Agrawal, Y. Yao, and C. W. Fletcher, “Morph: Flexible acceleration for 3d cnn-based video understanding,” in *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2018, pp. 933–946.
 - [51] X. Wei, Y. Liang, and J. Cong, “Overcoming data transfer bottlenecks in fpga-based dnn accelerators via layer conscious memory management,” in *2019 56th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2019, pp. 1–6.
 - [52] S. Sah and V. G. Vaidya, “Dependency aware ahead of time static scheduler for multicore,” in *2014 IEEE/ACIS 13th International Conference on Computer and Information Science (ICIS)*. IEEE, 2014, pp. 337–342.
 - [53] J. Qiu, J. Wang, S. Yao, K. Guo, B. Li, E. Zhou, J. Yu, T. Tang, N. Xu, S. Song *et al.*, “Going deeper with embedded fpga platform for convolutional neural network,” in *Proceedings of the 2016 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, 2016, pp. 26–35.
 - [54] A. Azizimazreah and L. Chen, “Shortcut mining: Exploiting cross-layer shortcut reuse in dcnn accelerators,” in *2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2019, pp. 94–105.
 - [55] Y. Guan, G. Sun, Z. Yuan, X. Li, N. Xu, S. Chen, J. Cong, and Y. Xie, “Crane: Mitigating accelerator under-utilization caused by sparsity irregularities in cnns,” *IEEE Transactions on Computers*, 2020.
 - [56] H. Kwon, P. Chatarasi, M. Pellauer, A. Parashar, V. Sarkar, and T. Krishna, “Understanding reuse, performance, and hardware cost of dnn dataflow: A data-centric approach,” in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 2019, pp. 754–768.
 - [57] A. Zela, T. Elsken, T. Saikia, Y. Marrakchi, T. Brox, and F. Hutter, “Understanding and robustifying differentiable architecture search,” *arXiv preprint arXiv:1909.09656*, 2019.
 - [58] M. Tan, B. Chen, R. Pang, V. Vasudevan, M. Sandler, A. Howard, and Q. V. Le, “Mnasnet: Platform-aware neural architecture search for mobile,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2019, pp. 2820–2828.
 - [59] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, “Rethinking the inception architecture for computer vision,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 2818–2826.
 - [60] A. Samajdar, Y. Zhu, P. Whatmough, M. Mattina, and T. Krishna, “Scale-sim: Systolic cnn accelerator simulator,” *arXiv preprint arXiv:1811.02883*, 2018.
 - [61] X. Lin, S. Yin, F. Tu, L. Liu, X. Li, and S. Wei, “Lcp: A layer clusters paralleling mapping method for accelerating inception and residual networks on fpga,” in *2018 55th ACM/ESDA/IEEE Design Automation Conference (DAC)*. IEEE, 2018, pp. 1–6.
 - [62] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, “Mobilenets: Efficient convolutional neural networks for mobile vision applications,” *arXiv preprint arXiv:1704.04861*, 2017.
 - [63] J. Fang, Y. Shen, Y. Wang, and L. Chen, “Optimizing dnn computation graph using graph substitutions,” *Proceedings of the VLDB Endowment*, vol. 13, no. 12, pp. 2734–2746, 2020.
 - [64] B. H. Ahn, J. Lee, J. M. Lin, H.-P. Cheng, J. Hou, and H. Esmaeilzadeh, “Ordering chaos: Memory-aware scheduling of irregularly wired neural networks for edge devices,” *arXiv preprint arXiv:2003.02369*, 2020.
 - [65] Z. Jia, J. Thomas, T. Warszawski, M. Gao, M. Zaharia, and A. Aiken, “Optimizing dnn computation with relaxed graph substitutions,” in *Proceedings of the 2nd Conference on Systems and Machine Learning (SysML’19)*, 2019.
 - [66] S.-C. Kao, G. Jeong, and T. Krishna, “Confucius: Autonomous hardware resource assignment for dnn accelerators using reinforcement learning,” *arXiv preprint arXiv:2009.02010*, 2020.
 - [67] N. Srivastava, H. Jin, S. Smith, H. Rong, D. Albonesi, and Z. Zhang, “Tensaurus: A versatile accelerator for mixed sparse-dense tensor computations,” in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2020, pp. 689–702.
 - [68] X. Yang, M. Gao, Q. Liu, J. Setter, J. Pu, A. Nayak, S. Bell, K. Cao, H. Ha, P. Raina *et al.*, “Interstellar: Using halide’s scheduling language to analyze dnn accelerators,” in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2020, pp. 369–383.
 - [69] H. Sharma, J. Park, N. Suda, L. Lai, B. Chau, V. Chandra, and H. Esmaeilzadeh, “Bit fusion: Bit-level dynamically composable architecture for accelerating deep neural networks,” in *Proceedings of the 45th Annual International Symposium on Computer Architecture*. IEEE Press, 2018, pp. 764–775.
 - [70] K. Hegde, J. Yu, R. Agrawal, M. Yan, M. Pellauer, and C. Fletcher, “Ucnn: Exploiting computational reuse in deep neural networks via weight repetition,” in *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2018, pp. 674–687.
 - [71] V. Akhlaghi, A. Yazdanbakhsh, K. Samadi, R. K. Gupta, and H. Esmaeilzadeh, “Snapea: Predictive early activation for reducing computation in deep convolutional neural networks,” in *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2018, pp. 662–673.
 - [72] Y. S. Shao, J. Clemons, R. Venkatesan, B. Zimmer, M. Fojtik, N. Jiang, B. Keller, A. Klinefelter, N. Pinckney, P. Raina *et al.*, “Simba: Scaling deep-learning inference with multi-chip-module-based architecture,” in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 2019, pp. 14–27.
 - [73] C. Ding, S. Liao, Y. Wang, Z. Li, N. Liu, Y. Zhuo, C. Wang, X. Qian, Y. Bai, G. Yuan *et al.*, “Circnn: accelerating and compressing deep neural networks using block-circulant weight matrices,” in *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture*. ACM, 2017, pp. 395–408.
 - [74] H. Sharma, J. Park, D. Mahajan, E. Amaro, J. K. Kim, C. Shao, A. Mishra, and H. Esmaeilzadeh, “From high-level deep neural models to fpgas,” in *The 49th Annual IEEE/ACM International Symposium on Microarchitecture*. IEEE Press, 2016, p. 17.

- [75] E. Baek, D. Kwon, and J. Kim, "A multi-neural network acceleration architecture," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2020, pp. 940–953.
- [76] Y. Choi and M. Rhu, "Prema: A predictive multi-task scheduling algorithm for preemptible neural processing units," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2020, pp. 220–233.
- [77] X. Wang, R. Hou, B. Zhao, F. Yuan, J. Zhang, D. Meng, and X. Qian, "Dnnguard: An elastic heterogeneous dnn accelerator architecture against adversarial attacks," in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2020, pp. 19–34.
- [78] X. Hu, L. Liang, L. Deng, Y. Ji, Y. Ding, Z. Du, Q. Guo, T. Sherwood, Y. Xie *et al.*, "A systematic view of leakage risks in deep neural network systems," 2021.
- [79] Y. Gan, Y. Qiu, J. Leng, M. Guo, and Y. Zhu, "Ptolemy: Architecture support for robust deep learning," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 241–255.
- [80] S. Gupta and B. Akin, "Accelerator-aware neural network design using automl," *arXiv preprint arXiv:2003.02838*, 2020.
- [81] W. Jiang, X. Zhang, E. H.-M. Sha, L. Yang, Q. Zhuge, Y. Shi, and J. Hu, "Accuracy vs. efficiency: Achieving both through fpga-implementation aware neural architecture search," in *Proceedings of the 56th Annual Design Automation Conference 2019*, 2019, pp. 1–6.
- [82] L. Yang, W. Jiang, W. Liu, H. Edwin, Y. Shi, and J. Hu, "Co-exploring neural architecture and network-on-chip design for real-time artificial intelligence," in *2020 25th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 2020, pp. 85–90.
- [83] W. Chen, Y. Wang, S. Yang, C. Liu, and L. Zhang, "You only search once: A fast automation framework for single-stage dnn/accelerator co-design," *arXiv preprint arXiv:2005.07075*, 2020.
- [84] M. S. Abdelfattah, Ł. Dudziak, T. Chau, R. Lee, H. Kim, and N. D. Lane, "Best of both worlds: Automl codesign of a cnn and its hardware accelerator," *arXiv preprint arXiv:2002.05022*, 2020.
- [85] H. Cai, L. Zhu, and S. Han, "Proxylessnas: Direct neural architecture search on target task and hardware," *arXiv preprint arXiv:1812.00332*, 2018.